

PREMISE SELECTION IS NOT ENOUGH: ACTION-CONDITIONAL EVIDENCE ALLOCATION FOR LEAN

Anonymous authors

Paper under double-blind review

ABSTRACT

Premise selection is usually treated as a ranking problem: choose useful lemmas, pass them to a Lean prover, and hope reconstruction succeeds. We argue that this view is missing a second intervention. A selected name must still be compiled into the proof interface that can consume it: Hammer facts, HammerCore inputs, simp lemmas, local elimination facts, Aesop fact rules, or Aesop simp rules. We introduce ACE, a typed evidence compiler for *action-conditional evidence allocation* in Lean. The core finding is counterintuitive but repeatable: once the candidate names are fixed, small typed portfolios can recover nearly all tested oracle headroom, while learned routers do not yet beat the fixed typed control. In a Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness, the mixed oracle-core/retrieved action grid solves 58/230 kernel-verified goals versus 38/230 for the best single action; a four-retry fixed portfolio reaches 57/230 out of fold. Removing traced proof-core names still leaves a retrieved-only compiler effect: empty Hammer solves 29/230, the best standalone action solves 35/230, and a fixed portfolio reaches the 52/230 typed oracle. To check that the effect is not only a post-hoc fit to the 230-goal matrix, we freeze the retrieved-only four-action portfolio and evaluate two disjoint fresh folds: on 432 replayable holdout goals, empty Hammer solves 30, the best singleton solves 54, the frozen K=4 portfolio solves 67, and the tested-action oracle is 68. Exact Aesop controls further constrain the mechanism: joint oracle-core plus retrieved exposure is best at 38/230, but identity and simp-only controls reach 36/230 and 35/230, so the supported claim is source/interface sensitivity rather than a large pure facts-versus-simps complementarity. ACE is therefore not a new retriever or an adaptive-router victory; it is evidence that premise selection is incomplete until selected names are typed into executable Lean proof actions.

1 INTRODUCTION

Lean hammers and neural theorem provers depend on premise selection: given a proof goal, the system must choose accessible lemmas that make proof search tractable and reconstructable in Lean. Modern systems therefore invest heavily in retrieving better premises before running a backend prover (Yang et al., 2023; Zhu et al., 2025; Gao et al., 2026). This framing is necessary but incomplete. A selected Lean name is not yet an executable proof-search intervention; it must be represented for the tactic that will consume it. The same name may be useful as a Hammer fact, inert as a simp lemma, harmful as an Aesop rule, or useful only after being paired with a different interface.

This interface decision is easy to overlook because it is not the same as ordinary tactic selection. We are not only choosing whether to call Hammer, Aesop, or simplification. We are choosing how a fixed source of evidence should enter each proof mechanism: as facts, simp rules, HammerCore inputs, local elimination facts, or multi-channel Aesop programs. Under bounded search, more evidence is also not monotonic. Additional names can improve proof-core coverage while increasing branching, timeouts, or reconstruction ambiguity. Thus a high-recall premise list can still fail if it is compiled into the wrong Lean interface.

We propose ACE, an action-conditional evidence allocator for bounded Lean proof search. Given a replayable theorem context and a candidate set, ACE compiles selected names into typed proof actions and checks each action in Lean. Every comparison starts from the same empty-premise

Hammer attempt and then spends a fixed retry budget. This protocol turns the main question into an auditable intervention study: after names have been proposed, how much verified headroom comes from assigning them to the right proof interface?

The empirical answer is surprisingly strong for simple typed portfolios. In a Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness, we identify 230 replayable goals from 490 cleaned traced-corpus goals. A mixed source pool of traced oracle-core names and learned retrieved candidates gives a typed action oracle of 58/230, compared with 38/230 for the best single action. A four-retry fixed typed portfolio after the empty-Hammer baseline reaches 57/230 out of fold and 58/230 when fit on all training data. Homogeneous $K=4$ controls are much weaker: Aesop-only reaches 49/230, HammerCore-only and simplification-only reach 41/230, and Hammer-only reaches 32/230. The effect is therefore typed diversity, not simply more calls to one backend.

We then remove the most obvious alternative explanation: traced proof-core assistance. In a retrieved-only anchor on the same 230 replayable goals, empty Hammer solves 29/230 and the best standalone action solves 35/230, while a fixed typed portfolio reaches the 52/230 typed oracle out of fold. To test whether this is a post-hoc peculiarity of the 230-goal matrix, we freeze the retrieved-only four-action portfolio and evaluate two fresh disjoint folds with zero overlap against the original goals. Across 432 replayable holdout goals, empty Hammer solves 30, the best singleton solves 54, $K=3$ and $K=4$ solve 67, and the tested-action oracle is 68. This does not make ACE a full theorem-proving leaderboard system, but it does give prospective evidence that typed compilation survives beyond the matrix used to discover it.

The mechanism is also narrower than our early experiments suggested. Aesop with oracle-core plus retrieved evidence exposed jointly as facts and simp lemmas proves 38/230 goals, compared with 29/230 for empty Aesop. Exact controls show that this is not a large isolated facts-versus-simps complementarity: identity exposure proves 36/230, simps-only proves 35/230, facts-only proves 32/230, retrieved-only joint exposure proves 34/230, and oracle-core-only joint exposure proves 35/230. The robust readout is that selected evidence has source-dependent and interface-dependent behavior; the paper does not need the stronger channel-complementarity claim.

This paper makes three contributions. First, we formulate action-conditional evidence allocation as a distinct layer after premise retrieval: selected names must be typed into Lean proof interfaces, not only ranked. Second, we implement ACE in a real Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness and show that fixed typed portfolios recover nearly all tested oracle headroom in a mixed-source matrix, a retrieved-only anchor, and frozen fresh-holdout folds. Third, we report the boundaries that keep the claim honest: broad insertion can hurt, strict syntactic filtering does not explain the gains, exact Aesop controls weaken a tempting complementarity story, and logistic/ComplementNB allocators do not yet beat the fixed typed control.

The scope is deliberately narrower than a full theorem-proving leaderboard. The retrieved-only and fresh-holdout experiments test the downstream compiler once candidate names are fixed; they are not official reproductions of LeanSearch v2 or another complete global retrieval system. We also distinguish bridge replay from open-ended proof generation. These boundaries sharpen rather than weaken the claim: premise selection is incomplete until selected evidence is compiled into the Lean interface that can actually use it.

2 RELATED WORK

Premise selection and hammers. Premise selection is a long-standing interface between theorem provers and large formal libraries. LeanDojo introduced retrieval-augmented theorem proving infrastructure in Lean (Yang et al., 2023); recent Lean-hammer work studies learned premise ranking and ATP integration (Zhu et al., 2025); and LeanSearch v2 focuses on global Lean 4 premise retrieval (Gao et al., 2026). These systems motivate our setting: a retriever proposes useful names before proof search. ACE studies the next decision after retrieval: how those names should be represented for Lean’s consuming interface.

Proof-action control and feedback. Tactic-selection systems such as TacticToe choose tactics or tactic sequences (Gauthier et al., 2021), while recent Lean systems use compiler or verifier feedback for proof repair, agentic search, or process supervision (Wang et al., 2026; Ospanov et al., 2025; Ma

et al., 2026; Kim & Yun, 2026; Huang et al., 2025; Hu et al., 2024). ACE is narrower: it does not edit proof text or propose new theorem-level plans. It fixes a candidate evidence source and tests the typed action by which selected names enter Hammer, HammerCore, simplification, local elimination, and Aesop.

3 PROBLEM FORMULATION

Action-conditional evidence allocation under a fixed retry budget. Let g be a Lean goal and let $\mathcal{A}(g)$ be the accessible name set induced by imports, namespaces, declarations, and local context. A typed proof action is a pair $a = (\tau, P)$, where τ is a Lean interface such as Hammer, HammerCore, simplification, local elimination, or Aesop, and $P \subset \mathcal{A}(g)$ is the set of names exposed through that interface. More generally, a typed evidence program is an assignment

$$z_{p,\tau} \in \{0, 1\}, \quad \sum_{p \in \mathcal{A}(g)} z_{p,\tau} \leq b_\tau,$$

where $z_{p,\tau} = 1$ means that name p is exposed to interface channel τ under channel budget b_τ . The same name may receive multiple interface labels: an Aesop joint facts+simps action is one compiled program, not two independent retries. A compiler $C(z)$ maps this assignment to Lean tactic syntax, such as Hammer facts, HammerCore inputs, simplifier lemmas, local elimination facts, Aesop safe facts, and Aesop simp rules. This separates four decisions that are often conflated: candidate provenance, channel eligibility, channel assignment, and outer action scheduling.

An evidence allocator chooses a sequence of typed programs,

$$a_t = (\tau_t, P_t), \quad |P_t| \leq k_t,$$

and runs each action under a search budget B_t . The prover returns either a kernel-verified success or a failure observation F_t , such as a timeout, missing premise signal, type mismatch, typeclass failure, failed simplification, or reconstruction failure. A closed-loop selector is a policy

$$a_{t+1} = \pi(g, a_{\leq t}, F_t, H_t),$$

where H_t includes previous attempt history and candidate features. The objective is to maximize verified success under a fixed total retry budget, rather than to maximize static premise recall at a single k . A fixed typed portfolio is one allocator; a learned action router is another. The experiments ask whether interface typing itself creates headroom before claiming that learned routing has solved the allocation problem.

Evaluation axes. We separate three notions that are often conflated. *Kernel-verified proof-action success* means that a patched Lean file containing a candidate proof action checks successfully. This is the main verified metric for ACE. *Trace-core success* means that the selected premise set covers the traced proof-core premises for a goal; it is a premise-recovery metric, not a full proving metric. *Bridge verified success* is stricter than trace-core success: trace-core recovery must coincide with successful replay of the original traced tactic script in a real Lean process. Bridge verification is still not a full proof-generation benchmark, because it replays known tactic scripts, but it filters out trace-core cases that drift too far from Lean execution.

4 METHOD

4.1 OVERVIEW

ACE treats a selected name as incomplete until it is assigned to a proof-action interface. Figure 1 gives the full loop. The system first builds a candidate pool and runs a shared empty-premise Hammer attempt. It then spends a fixed retry budget on typed actions that expose selected names as Hammer facts, HammerCore inputs, simp lemmas, local elimination facts, or Aesop facts/simp rules. The important design choice is that the retry budget is spent on compiling evidence into interfaces, not on blindly expanding a single untyped premise list.

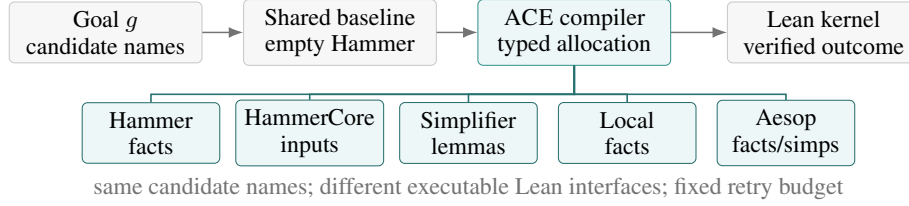


Figure 1: ACE evaluates action-conditional evidence allocation under a fixed retry budget. Selected names are compiled into typed Lean interfaces, and each routed action is checked by the Lean kernel.

4.2 TYPED EVIDENCE PROGRAMS

The allocator is built from action families that consume selected names differently. Hammer actions pass names as ordinary premises; HammerCore actions expose a smaller internal proof-search interface; simplification actions use selected names as rewrite/simp lemmas; local elimination uses names as local facts; Aesop actions can add names as fact rules, simp rules, or both. For each replayable theorem context, we materialize a patched pre-theorem Lean file for each action and classify the result by running Lean with a fixed timeout.

The fixed portfolio is chosen greedily on the training split and evaluated out of fold. Every policy receives the same first action, empty Hammer, and then a retry budget K . This protocol makes the strongest control explicit: an adaptive policy is only useful if it beats or compresses the best fixed sequence of typed actions under the same Lean-call budget. In the current matrix, the train-fitted four-action portfolio after empty Hammer combines Aesop, HammerCore, Hammer, and local-elimination actions.

This design also prevents a common misreading of the method. The fixed portfolio is not intended to prove that a hand-written order is optimal; it is the hard control that any learned action router must beat. Its near-oracle performance is therefore evidence that interface typing itself is a large effect, while the absence of an adaptive win is reported as a boundary rather than hidden as a negative result.

4.3 AESOP FACT/SIMP EXPOSURE

Aesop is the most important new interface in the verified matrix, so we isolate how selected names enter it. Given a ranked name list, we split available names into fact candidates and simp candidates, then instantiate Aesop in three exposure modes: facts-only, simps-only, and facts+simps. We also vary source pools: traced oracle-core names, retrieved-only names, and oracle-core plus retrieved names at budgets 8, 16, and 32. This gives a matched source/budget control: the goal and candidate source are fixed, while the channel assignment is removed, added, or widened.

The Aesop ablation is not a new oracle-improving action set; its role is explanatory. The original broad ablation suggested a strong facts+simps interaction, but exact matched controls give a more conservative readout. For the best current oracle-core plus retrieved source, joint exposure recovers the 38/230 Aesop result, yet identity exposure reaches 36/230, simps-only reaches 35/230, facts-only reaches 32/230, and joint-only over exact single-channel controls is only 2 goals. Retrieved-only joint exposure still reaches 34/230 and oracle-core-only reaches 35/230. The supported mechanism is therefore source/exposure/search sensitivity, not a clean causal claim that facts and simps are complementary in isolation.

4.4 WHY TYPED ACTIONS SHOULD HELP

Let $Y_g(z; B) \in \{0, 1\}$ denote the kernel-verified potential outcome for goal g when typed assignment z is checked under search budget B . The marginal intervention effect of exposing name p through interface τ is

$$\Delta_g(p, \tau \mid z, B) = Y_g(z + e_{p, \tau}; B) - Y_g(z; B).$$

This formulation is intentionally intervention-level rather than only a marginal retrieval score. It permits the same name to have positive effect in one interface and negative effect in another, allows facts and simp rules to interact through the current program z , and permits larger evidence sets to

have negative marginal effect under bounded search. A typed action helps when the compiler finds assignments whose realized intervention effects are positive under the Lean backend’s budget, not merely when a retriever ranks relevant names highly.

5 EXPERIMENTS

Protocol. All main results are kernel-verified Lean checks in a Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness. We first clean 490 traced-corpus goals and keep the 230 goals whose original tactic script replays before the theorem statement. We then run typed proof actions in Lean and count a success only when the patched file checks. To separate interface allocation from traced proof-core assistance, we repeat the budgeted compiler test with only retrieved candidates. Finally, we freeze the retrieved-only action subset and evaluate two disjoint 500-goal folds with zero overlap against the original 230-goal set; the folds contain 432 replayable goals in total. Trace-core and bridge experiments are retained in Appendix Table 15 as discovery evidence for bounded retries, not as the main proving metric.

Main replay results. The central comparison fixes the candidate source and changes only how names are compiled into Lean proof actions. Table 1 reports the effect in three increasingly strict settings: the mixed-source discovery matrix, a retrieved-only anchor that removes traced proof-core assistance, and a frozen fresh-holdout evaluation whose action subset and order are fixed before testing.

Table 1: Core verified evidence. “Portfolio” is the fixed typed retry sequence after the shared empty-Hammer call; the fresh-holdout row uses the frozen retrieved-only sequence and an oracle over the predeclared tested actions.

Setting	Empty	Best single	Portfolio	Oracle	Readout
Mixed-source 230	29/230	38/230	57/230 OOF	58/230	typed actions create +20 oracle headroom
Retrieved-only 230	29/230	35/230	52/230 OOF	52/230	removes traced proof-core assistance
Frozen fresh holdout	30/432	54/432	67/432	68/432	transfers beyond the discovery matrix

The three rows give the same qualitative answer. On the mixed-source matrix, the best single typed action solves 38 goals, while the tested-action oracle solves 58; a four-retry fixed portfolio reaches 57 out of fold. Removing traced proof-core names preserves the pattern: the best standalone retrieved-only action solves 35/230, but the fixed typed portfolio reaches the 52/230 tested-action oracle. The prospective holdout is the most conservative check. On 432 replayable unseen goals, the frozen K=4 sequence solves 67 goals, compared with 54 for the best singleton and 68 for the oracle. Figure 2 normalizes these numbers by each setting’s tested-action oracle, making the common recovery pattern visible without mixing denominators.

Budget and transfer. The main result would be weak if it required an unconstrained search over many proof actions. We therefore track performance after each retry slot under the same shared empty-Hammer first attempt. The budget path in Table 2 shows that most of the tested-action headroom is recovered within three or four typed calls, and Table 3 verifies that the frozen retrieved-only sequence behaves similarly on two disjoint holdout folds.

Table 2: Budget path after the shared empty-Hammer call. The same typed-portfolio story holds in the mixed-source matrix, the retrieved-only anchor, and the frozen holdout.

Setting	K=1	K=2	K=3	K=4	Oracle
Mixed-source 230	49/230	55/230	55/230	57/230	58/230
Retrieved-only 230	47/230	50/230	52/230	52/230	52/230
Frozen holdout 432	56/432	58/432	67/432	67/432	68/432

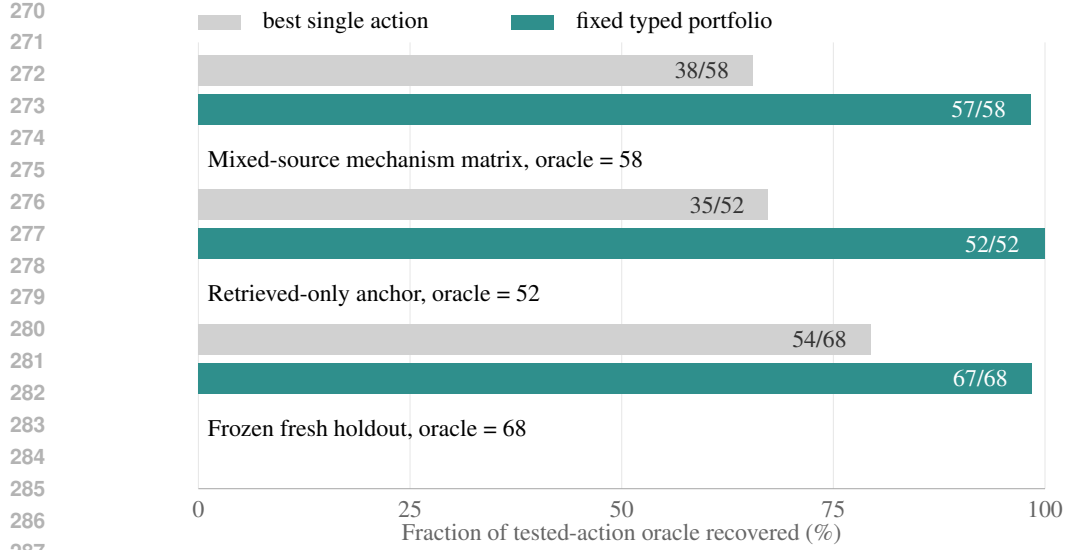


Figure 2: Oracle recovery across the three main evidence settings. Bars are normalized within each setting by the tested-action oracle. The same fixed typed-portfolio pattern appears in the mixed mechanism matrix, the retrieved-only deployability anchor, and frozen fresh-holdout folds.

Table 3: Fresh-holdout fold details. The action subset and order are frozen before evaluation; the two folds have zero overlap with the original 230-goal matrix.

Fold	Replayable	Empty	Best single	K=3	K=4	Oracle
Fold 0	208	16	23	28	28	28
Fold 1	224	14	31	39	39	40
Combined	432	30	54	67	67	68

The budget curves make the fixed sequence a strong baseline rather than a strawman. On the retrieved-only anchor, K=3 already reaches the tested-action oracle. On the frozen holdout, K=3 and K=4 recover 67/68 tested-action opportunities, and the strict after-empty-Hammer coverage is 37/38. The fold-level numbers are not identical, but both folds move in the same direction: the frozen typed sequence beats the strongest singleton and remains within one goal of the tested-action oracle. This is the evidence that the effect is not only a fit to the discovery matrix.

Action-level frontier. The typed portfolio only matters if the extra successes are not obtainable by simply choosing one dominant interface. Table 4 gives the leading single actions in the mixed-source grid and exposes the interface diversity behind the oracle gap.

Table 4: Leading single typed actions in the mixed-source 230-goal grid. The table exposes what the best singleton in Table 1 actually is and why a portfolio must cross interfaces.

Single typed action	Verified	Mean facts	Mean sims	Mean time (s)	Role in the evidence chain
Aesop, core + retrieved-8 as facts/sims	38/230	9.41	7.86	8.92	best singleton
Aesop, core + retrieved-16 as facts/sims	37/230	12.88	10.40	9.41	nearby Aesop variant
Hammer, core + retrieved-16 facts	32/230	12.88	0.00	8.56	strongest Hammer fact interface
Hammer, core + retrieved-32 facts	32/230	14.05	0.00	8.55	broader facts do not help
HammerCore, core + retrieved-8 inputs	18/230	9.41	0.00	13.29	useful but not dominant
Empty Hammer	29/230	0.00	0.00	8.31	shared first attempt

The best singleton is an Aesop joint-exposure action, but Hammer facts and HammerCore inputs remain useful in different parts of the oracle. Simply widening the same interface is also insufficient: Hammer retrieved-32 does not improve over Hammer retrieved-16, and the best singleton remains far below the 58-goal tested-action oracle. This is the local evidence for the paper’s abstraction: the

object being selected is not only a set of names, but a typed evidence program that assigns names to proof interfaces.

Control ablations. The remaining experiments rule out five weaker explanations for the main effect: homogeneous retrying, random portfolio order, simple learned routing, a special Aesop channel effect, and strict syntactic cleanup. Table 5 summarizes the headline controls; expanded matrices, runtime accounting, bridge logs, and split diagnostics remain in the appendix.

Table 5: Controls for typed allocation on the mixed-source matrix. Expanded versions, including call and wall-clock accounting, appear in the appendix.

Control axis	Reference	Strongest matched alternative	Main readout
Typed vs. homogeneous K=4	Fixed typed 57/230	Aesop-only 49/230; HammerCore 41/230; simp 41/230; Hammer 32/230	Cross-interface allocation matters.
Fixed order vs. random order	Fixed typed 57/230	Random full-grid K=4: mean 41.9, range 31–56 over 100 seeds	The selected order is not generic portfolio luck.
Fixed order vs. learned gates	Fixed typed 57/230	Residual logreg 57/230; ComplementNB 57/230; pure logreg 47/230	Current routers do not beat the static compiler.
Aesop channel assignment	Joint exposure 38/230	Identity 36/230; simps-only 35/230; facts-only 32/230	The Aesop gain is modest and source-sensitive.
Strict syntactic filtering	Full oracle 58/230	Filtered-only oracle 4/230; 0 new oracle goals	Filtering alone is not the mechanism.

These controls keep the main claim narrow. Homogeneous retries leave substantial headroom, random K=4 schedules do not match the selected fixed sequence, and simple learned gates match but do not improve on it. Exact Aesop controls further prevent an overclaim about facts-versus-simps synergy: joint exposure is best, but the margin over identity and simps-only controls is small. The supported conclusion is therefore not that one tactic channel is universally superior. It is that Lean proof search benefits when proposed names are compiled into a small typed portfolio rather than passed to one interface as an untyped ranked list.

6 THEORY AND ANALYSIS

Proposition 1 (A scalar premise score cannot represent typed intervention value). *Let $Y_g(z; B) \in \{0, 1\}$ be the kernel-verified outcome for goal g under typed evidence assignment z and search budget B . If there exist a name p and two interfaces τ_1, τ_2 such that*

$$Y_g(z + e_{p, \tau_1}; B) \neq Y_g(z + e_{p, \tau_2}; B),$$

then no interface-agnostic premise score for p can represent the full intervention effect of exposing p to Lean.

Sketch. The intervention $e_{p, \tau}$ changes both the syntax sent to Lean and the backend search process that consumes the name. If two interface assignments yield different verified outcomes under the same goal and budget, a premise ranking over p alone has discarded information needed to choose the successful action. This is the formal reason for compiling selected evidence into typed proof actions rather than treating retrieval as the final decision. Table 1 gives the empirical version: the best single action is far below the typed oracle, while a small typed portfolio recovers nearly all tested headroom.

Proposition 2 (Premise coverage is not monotonic in premise budget). *There can exist premise sets $P_1 \subset P_2$ such that P_1 covers enough useful premises for the backend to succeed under budget B , while P_2 contains the same useful premises but causes timeout or reconstruction failure under the same budget.*

Sketch. Adding premises preserves proof-core coverage but can increase branching, generated clauses, simplification search, or reconstruction ambiguity. Under fixed time/search budget, the prover may fail on the larger set even though the smaller set contains the same proof-relevant lemmas. The controls in Table 5 are consistent with this view: homogeneous retries, broad Aesop exposure, and strict syntactic filtering do not explain the gains, while typed diversity does.

7 SCOPE AND DEPLOYMENT NOTES

The experiments are designed as a proof-action intervention study rather than a leaderboard entry. The main matrix uses 230 replayable pre-theorem Mathlib contexts, and the frozen holdout adds 432 replayable retrieved-only contexts with no overlap against the discovery set. This setup avoids target-theorem leakage and makes each reported success a kernel-checked Lean result. A full external deployment would naturally combine ACE with stronger global retrievers such as LeanSearch-style systems; our retrieved-only rows study the downstream compiler once such candidates have been proposed.

The current strongest policy is intentionally simple: a fixed typed portfolio. This is a useful property for the present claim because it shows that much of the headroom comes from interface typing itself, before more complex adaptive routing is needed. Learned routers, trace-core controllers, wallclock frontiers, and bridge diagnostics are therefore reported as supporting analyses rather than as the paper’s central promise.

8 DISCUSSION

What changes for Lean premise systems? The practical implication is that a premise selector should not hand a prover only an ordered list of names. It should hand off a typed evidence program: which names are ordinary Hammer facts, which are HammerCore inputs, which are simplification lemmas, which are elimination facts, and which are Aesop fact or simp rules. This is a small interface change but a different abstraction boundary. Retrieval remains responsible for proposing useful names; the compiler layer decides how those names become executable proof actions.

Why the fixed portfolio is informative. The fixed portfolio is not presented as an optimal hand-written strategy. It is the strongest failure-agnostic control we found under the same outer retry budget. Its near-oracle behavior is exactly what makes the result interesting: much of the current headroom is recovered by crossing typed interfaces at all, before asking a learned router to decide per goal. This also sets a higher bar for future adaptive methods. A router should not be compared only to empty Hammer or to a single tactic; it should beat a compact typed portfolio.

A compiler checklist for future systems. The experiments suggest a concrete reporting discipline for retrieval-augmented Lean provers. A paper should report the candidate source separately from the proof-action interface, include a fixed typed portfolio before claiming an adaptive router is useful, add a retrieved-only compiler test when traced proof-core names appear during development, and report whether extra names improve backend success or only increase search cost. This checklist does not require a new benchmark or a new global retriever; it asks future systems to distinguish retrieval quality from interface compilation when a proposed name list fails.

Why this is not only a benchmark result. The main result is an intervention on the proof interface, not a new leaderboard score. The same candidate source can produce different verified outcomes depending on how names enter Lean. This creates a reusable diagnostic for future prover work: when a retriever appears weak, one should ask whether the missing step is retrieval quality or evidence compilation. Conversely, when a large premise set hurts, the remedy may be a different typed action rather than another reranking pass over the same untyped list.

How to read the remaining failures. The unsolved goals are also informative because the intervention protocol localizes them. A miss after every single typed action points to candidate quality; a miss after the full portfolio points to missing proof actions, longer reasoning chains, or backend search limits. Thus the oracle, homogeneous, strict-goal, and retrieved-only controls turn raw failures into a map of where the prover stack should improve next.

What a stronger next system should do. The natural next step is to learn an allocator that beats the fixed typed portfolio while preserving the same accounting discipline: same first attempt, same retry budget, same verified-outcome metric, and the same strong static controls. The present experiments identify where the oracle headroom is, show that a simple typed sequence captures most of it, and separate the compiler effect from both traced proof-core assistance and post-hoc portfolio fitting.

Why the evidence chain is specific. The strongest part of the empirical story is not any single number in isolation, but the sequence of exclusions. The mixed-source matrix establishes that interface assignment creates verified headroom after candidate names are available. The retrieved-only anchor removes the easiest alternative explanation, namely that the effect is carried by traced proof-core names. The frozen holdout then tests the same action subset outside the discovery matrix. Finally, the control ablations show that the result is not simply more retries, not a random portfolio, not an adaptive-router win, and not a pure facts-versus-simps story. This is why the paper presents typed evidence allocation as the main abstraction rather than as a collection of tactic tricks.

Scope of the claim. The paper does not claim to replace global premise retrieval or to solve open-ended theorem proving. Its claim is downstream and interface-level: once a retriever, trace, or human source proposes names, the system still needs to decide how those names enter Lean. That scope is also what makes the result portable. Better retrievers should improve the candidate pool; ACE asks whether the resulting names should be passed as Hammer facts, HammerCore inputs, simplification lemmas, local facts, or Aesop rules under the same verified-budget accounting.

Where the appendix evidence fits. The appendix is organized by evidence role rather than by experiment chronology. The corpus and replay tables define which goals can be counted. The full action matrix and provenance tables show that the singleton frontier in the main text is not hiding an omitted dominant action. The homogeneous, random-order, router, exact-Aesop, and strict-filtering controls expand the compressed main ablations. The bridge and trace-core diagnostics explain how the search space was discovered while keeping the main proving metric tied to replayable Lean checks. This organization matters because the appendix is not meant to create a second story; it supplies the audit trail for the same typed-allocation claim.

A stronger baseline for future work. The empirical bar set by this paper is intentionally static. A future adaptive allocator should be evaluated only after it beats the fixed typed portfolio under the same empty-Hammer first attempt, retry budget, candidate source, and kernel-verified outcome metric. This changes the comparison target for retrieval-augmented Lean systems. Improving over empty Hammer or over a single tactic is no longer enough once a small typed portfolio recovers most of the tested oracle; the next useful improvement must decide when the static sequence is wrong and recover the remaining goals without spending an unaccounted search budget.

9 CONCLUSION

This paper argues that Lean premise selection is incomplete without action-conditional evidence allocation. ACE implements this as a typed compiler over Hammer, HammerCore, simplification, local-elimination, and Aesop interfaces. Across the verified replay harness, typed allocation closes most tested-oracle headroom: 57/58 on mixed-source, 52/52 retrieved-only, and 67/68 on fresh holdout. Homogeneous retries are weaker, learned gates do not beat the fixed sequence, and exact Aesop controls point to source/interface sensitivity. The lesson is narrow but reusable: a retriever should hand Lean a typed evidence program, not only a ranked name list, because the same evidence can help or hurt depending on its consuming proof interface.

REFERENCES

- Guoxiong Gao, Zeming Sun, Jiedong Jiang, Yutong Wang, Jingda Xu, Peihao Wu, Bryan Dai, and Bin Dong. Leansearch v2: Global premise retrieval for lean 4 theorem proving. *arXiv preprint arXiv:2605.13137*, 2026. URL <https://arxiv.org/abs/2605.13137>.
- Thibault Gauthier, Cezary Kaliszyk, Josef Urban, Ramana Kumar, and Michael Norrish. Tactictoe: Learning to prove with tactics. *Journal of Automated Reasoning*, 65:257–286, 2021. URL <https://arxiv.org/abs/1804.00596>.
- Edward J. Hu et al. minictx: Neural theorem proving with long contexts. *arXiv preprint arXiv:2408.03350*, 2024. URL <https://arxiv.org/abs/2408.03350>.
- Suozhi Huang, Peiyang Song, Robert Joseph George, and Anima Anandkumar. Leanprogress: Guiding search for neural theorem proving via proof progress prediction. *arXiv preprint arXiv:2502.17925*, 2025. URL <https://arxiv.org/abs/2502.17925>.
- Minsu Kim and Se-Young Yun. Process-verified reinforcement learning for theorem proving via lean. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=P00k4DFaXF>.
- David Ma, Kaijing Ma, Shawn Guo, Yunfeng Shi, Enduo Zhao, Jiajun Shi, Zhaoxiang Zhang, Gavin Cheung, Jiaheng Liu, and Zili Wang. Oprover: A unified framework for agentic formal theorem proving. *arXiv preprint arXiv:2605.17283*, 2026. URL <https://arxiv.org/abs/2605.17283>.
- Azim Ospanov, Farzan Farnia, and Roozbeh Yousefzadeh. Apollo: Automated llm and lean collaboration for advanced formal reasoning. In *Advances in Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=fxDCgOruk0>.
- Evan Wang, Simon Chess, Daniel Lee, Siyuan Ge, Ajit Mallavarapu, and Vasily Il'in. Learning to repair lean proofs from compiler feedback. *arXiv preprint arXiv:2602.02990*, 2026. URL <https://arxiv.org/abs/2602.02990>.
- Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2306.15626>.
- Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. Premise selection for a lean hammer. *arXiv preprint arXiv:2506.07477*, 2025. URL <https://arxiv.org/abs/2506.07477>.

A ADDITIONAL EXPERIMENTAL EVIDENCE

A.1 CORPUS CONSTRUCTION AND REPLAY CHECKS

Table 6 spells out the replay filtering behind the verified experiments. The main 230-goal matrix is drawn from the 490 clean traced-corpus goals, but every reported success is rechecked by Lean after patching the theorem context. The fresh-holdout folds are disjoint from the 230-goal matrix and use the frozen retrieved-only action subset.

Table 6: Corpus construction, cleaning, and replayability. “Replayable” means that the original proof script or held-out theorem context checks in the pre-theorem harness before any ACE intervention is counted.

Evaluation source	Input	Clean	Dropped	Replayable	Main use
Traced subset cleaning	500	490	10	—	source pool for 230-goal matrix
Replay audit sample	300	—	—	143	failure taxonomy sanity check
Fresh fold 0	500	483	17	208	frozen retrieved-only holdout
Fresh fold 1	500	483	17	224	frozen retrieved-only holdout
Fresh folds combined	1000	966	34	432	prospective transfer check

The 10 dropped traces in the 500-goal cleaning pass consist of 3 missing files, 6 missing theorem declarations, and 1 missing proof-core record. For the 300-goal replay audit, the major non-replayable categories are 59 unknown identifiers, 36 tactic failures, 22 type mismatches, 11 parse errors, 9 typeclass or inference failures, 8 rewrite failures, 7 invalid source spans, 3 simplifier failures, and 2 missing proof markers.

A.2 FULL ACTION MATRIX AND EVIDENCE PROVENANCE

The main paper reports the best singleton, portfolio, and oracle values. Table 7 gives the leading individual proof actions from the mixed-source grid. The important pattern is that several typed interfaces are viable, but none alone closes the oracle gap.

Table 7: Leading single actions in the mixed-source 230-goal grid. Mean facts and simp rules are measured over attempted goals for that action.

Single action	Verified	Mean facts	Mean_simps	Mean time (s)	Interface readout
Aesop, core + retrieved-8 as facts/simps	38/230	9.41	7.86	8.92	best singleton
Aesop, core + retrieved-16 as facts/simps	37/230	12.88	10.40	9.41	similar Aesop channel
Hammer, core + retrieved-16 facts	32/230	12.88	0.00	8.56	strong Hammer fact interface
Hammer, core + retrieved-32 facts	32/230	14.05	0.00	8.55	no gain from broader list
Hammer, core + retrieved-8 facts	31/230	9.41	0.00	8.67	compact Hammer fact interface
Aesop with no supplied evidence	29/230	0.00	0.00	7.99	empty-action baseline
Hammer with no supplied evidence	29/230	0.00	0.00	8.31	shared first attempt
HammerCore, core + retrieved-8 inputs	18/230	9.41	0.00	13.29	useful but not dominant

Table 8: Candidate provenance in the 230-goal matrix. The retrieved lists overlap with traced proof-core names but do not contain all of them, which is why the retrieved-only anchor is reported separately.

Candidate statistic	Total names	Mean per goal	Median per goal
Traced proof-core names	875	3.80	3
Retrieved top-8 names	1840	8.00	8
Retrieved top-32 names	7360	32.00	32
Proof-core names inside top-8	552	2.40	—
Proof-core names inside top-32	738	3.21	—
Proof-core names outside top-32	137	0.60	—

A.3 BUDGET, HOMOGENEOUS CONTROLS, AND COMPUTE

Table 9 expands the main homogeneous-control result with call and wall-clock accounting. The full typed grid is not merely spending more calls: every homogeneous K=4 family uses a comparable retry budget but leaves substantial verified headroom.

Table 9: Homogeneous K=4 controls on the mixed-source 230-goal grid. Average calls and wall-clock include the shared empty attempt and subsequent typed retries.

K=4 policy family	Verified	Avg calls	Avg wall-clock (s)
Full typed action grid	57/230	4.18	39.08
Aesop-only	49/230	4.23	28.06
HammerCore-only	41/230	4.34	50.71
Simplification-only	41/230	4.34	35.41
Local elimination-only	34/230	4.43	41.32
Hammer-only	32/230	4.47	38.89
Raw arithmetic/normalization-only	29/230	4.50	34.77
Random full-grid K=4, 100 seeds	41.9/230 mean	—	—

For all-training compute diagnosis, the fitted full-grid frontier has average calls 1.87, 2.66, 3.42, and 4.18 for K=1 through K=4, with average wall-clock 16.36s, 26.37s, 33.12s, and 40.07s. At K=4 the

median is 30.71s and the 90th percentile is 86.40s. The first-success leaders are empty Hammer on 29 goals, Aesop core + retrieved evidence on 20 additional goals, and HammerCore core + retrieved evidence on 6 additional goals.

A.4 RETRIEVED-ONLY ANCHOR

The retrieved-only anchor removes traced proof-core names from the typed evidence pool. Table 10 shows that the same typed-portfolio pattern survives: the fixed sequence closes the entire tested-action oracle out of fold.

Table 10: Retrieved-only anchor on the 230-goal matrix. Strict goals are the after-empty goals that require action-dependent evidence rather than the shared empty attempt.

Readout	Verified goals	Strict after-empty goals
Empty Hammer	29/230	–
Best standalone retrieved-only action	35/230	–
K=1 fixed portfolio	47/230	18/23
K=2 fixed portfolio	50/230	21/23
K=3 fixed portfolio	52/230	23/23
K=4 fixed portfolio	52/230	23/23
Typed action oracle	52/230	23/23

A.5 FROZEN FRESH-HOLDOUT DETAILS

The fresh-holdout experiment freezes the retrieved-only action subset before evaluating two disjoint 500-goal folds. Table 11 gives the per-fold result rather than only the combined main-table number.

Table 11: Frozen retrieved-only portfolio on two disjoint fresh folds. The combined row is the prospective 432-goal result reported in the main paper.

Fold	Replayable	Empty	Best single	K=1	K=2	K=3	K=4	Oracle
Fold 0	208	16	23	24	24	28	28	28
Fold 1	224	14	31	32	34	39	39	40
Combined	432	30	54	56	58	67	67	68

The frozen four-action subset is Aesop core + retrieved-16, HammerCore core + retrieved, Aesop core + retrieved with swapped exposure, and Aesop core. On the combined holdout, 37 of 38 strict after-empty goals are covered by K=3 and K=4.

A.6 AESOP SOURCE AND CHANNEL CONTROLS

The broad Aesop ablation initially suggested a strong facts-versus-simps story, but exact controls narrowed that interpretation. Table 12 gives the broad readout; Table 13 gives the cleaner matched-control version that the main paper relies on.

Table 12: Broad Aesop ablation on the 230-goal matrix. Joint core + retrieved exposure is best, but this broad sweep alone does not prove a large pure channel-complementarity effect.

Aesop variant	Verified
Core + retrieved-8 as facts and sims	38/230
Core + retrieved-16 as facts and sims	37/230
Empty Aesop	29/230
Facts-only variants, best	5/230
Core-only or sims-only variants, best	4/230
Retrieved-only channel variants, best	4/230

Table 13: Exact Aesop controls. The supported claim is source/interface sensitivity: joint exposure is useful, but identity and simps-only controls are close enough that the paper should not overclaim a large isolated facts-versus-simps complementarity.

Evidence source	Joint	Identity	Swapped	Simps-only	Facts-only	Random split	Joint-only gain
Oracle core only	35	35	35	33	31	32	1
Oracle core + retrieved	38	36	33	35	32	32	2
Retrieved only	34	31	32	32	30	29	1
Empty Aesop	29	–	–	–	–	–	–

A.7 STRICT FILTERING AND CANDIDATE AUDIT

The strict syntactic filter tests whether the gains are explained by simply removing unsafe or non-theorem-like names. Table 14 shows that this is not the mechanism: filtered-only actions add essentially no oracle headroom and often lose the successful source action.

Table 14: Strict interface filtering boundary. “Source” is the corresponding unfiltered action; “kept”, “new”, and “lost” compare verified goals against that source action.

Filtered action	Filtered	Source	Kept	New	Lost
Aesop core + retrieved-16 filtered	3	37	1	2	36
Aesop core + retrieved-8 filtered	3	38	1	2	37
Hammer core + retrieved-16 filtered	3	32	3	0	29
HammerCore core + retrieved filtered	1	18	1	0	17

At the oracle level, the unfiltered baseline solves 58/230, the filtered-only oracle solves 4/230, and the combined oracle remains 58/230 with 0 new oracle goals from the filtered-only set. The candidate audit counts 7321 available names, 7360 retrieved candidate placements, 875 proof-core names, 7497 selected placements, 176 unavailable placements, 3268 theorem-like names, 1927 names carrying simplifier attributes, 1880 definition-like names, 161 class-like names, and 137 proof-core-only names outside the retrieved top-32.

A.8 TRACE-CORE DISCOVERY SUMMARY

Table 15: Representative trace-core discovery results that motivated the verified typed-action protocol. These rows are not the main proving metric; they explain the design pressure for bounded retries, non-monotonic premise budgets, and bridge checks.

Discovery question	Strongest useful result	Readout for current paper
Can failure feedback improve local premise recovery?	Local trace-core FAR-HAMMER reaches 95.7% trace success and 72.0% bridge verified success; blind top- k expansion reaches 91.5% trace and 43.0% bridge.	Failure events can change which evidence is useful, but bridge verification is stricter than trace-core recovery.
Can adding more premises hurt?	Timeout-conditioned shrink reaches 69.3% success with 23.5% timeout rate, while equal-budget top- k falls to 43.8% with 56.2% timeouts.	Premise coverage is not monotonic under bounded search, motivating typed budgets instead of broad insertion.
Do local heuristics solve global imported premises?	Imported-core lexical/BM25 stress stays near 12% success before learned retrieval.	Local feature control is insufficient for global retrieval; the verified paper should not claim full retriever replacement.
Does learned imported retrieval survive bridge checks?	Final-base8 averages 92.9% across four splits and reaches 53.5% imported bridge verified success versus 44.5% for learned+base fallback.	Bridge evidence supports the premise-recovery story but remains separate from open-ended proof generation.

Table 16: Feature-group ablation on 2k local trace-core goals. Static visible-feature reranking is weaker than blind expansion; failure-conditioned feature use is the important step.

Method	Success	FFR	Avg prem.
Top- k expansion	91.5%	79.9%	46.9
Visible-feature name rerank	85.3%	0.0%	42.9
Visible-feature statement rerank	77.6%	0.0%	42.9
Visible-feature declaration rerank	78.3%	0.0%	42.9
Visible-feature rerank	86.5%	0.0%	42.9
Rule FAR, no core declaration features	92.6%	82.4%	46.2
Rule FAR, no core statement features	92.8%	82.9%	46.4
Rule FAR, no core name features	95.0%	88.1%	45.4
Rule FAR, no core tags	95.7%	89.7%	45.1

Table 17: Initial imported-core stress test on 2k goals. This is a diagnostic negative result: local and lexical signals are insufficient for global imported-premise recovery.

Method	Success	FFR	Avg prem.
One-shot	1.1%	0.0%	32.0
Same-file prior rerank	2.1%	0.0%	56.0
Visible-feature rerank	2.4%	0.0%	56.0
BM25 rerank	7.0%	0.0%	56.0
LeanSearch iterative	11.9%	8.3%	91.9
BM25 expansion	11.9%	8.3%	92.2
Rule FAR + BM25	12.2%	8.7%	92.2
Rule FAR full, oracle reference	55.7%	55.2%	86.6

A.9 BRIDGE REPLAY AND FAILURE TAXONOMY

The 200-goal imported bridge verifies 124/200 original traced tactic scripts. Table 18 gives the policy-level bridge result; Table 19 gives the replay-failure distribution.

Table 18: Imported bridge policy-level diagnostics on 200 goals. Trace-core success and bridge-verified success are intentionally reported separately.

Policy	Trace success	Bridge verified	Replay-verified trace misses	Avg prem.
Learned expansion	60.0%	38.5%	47	72.8
Learned + base fallback	67.5%	44.5%	35	69.9
Rule FAR learned second stage	81.0%	52.5%	19	66.2
Final base guardrail-8	81.5%	53.5%	17	66.2

Table 19: Primary replay outcome taxonomy on the 200-goal imported bridge sample.

Replay outcome	Count
Replay verified	124
Unknown identifier	30
Other Lean error	13
Typeclass or inference failure	8
Unsolved goals	7
Simplifier no progress	6
Tactic failed	5
Type mismatch	5
Sorry context	2

Negative-case inspection of the 17 final replay-verified misses finds 0 candidate-pool misses, 2 dropped fallback-selected core premises, 3 dropped expansion-selected core premises, and 15 failure-conditioned rank misses. The best second-stage rank of the missed core premise has median 117, minimum 94, maximum 207, and only 2/17 appear inside the top 96. These failures point to ranking and calibration rather than missing candidate generation.

A.10 SPLIT STABILITY AND NEGATIVE EXPERT-MIXING RESULTS

Table 20: Split stability for the imported trace-core controller. These are discovery diagnostics, not the main verified proving metric.

Split	Fallback	Second stage	Final guardrail
Original	87.0%	92.4%	92.6%
Fold 0	83.6%	91.6%	91.6%
Fold 1	86.0%	92.2%	93.4%
Fold 2	88.8%	94.0%	94.0%
Mean $\pm$ std	86.3 $\pm$ 1.9	92.5 $\pm$ 0.9	92.9 $\pm$ 0.9

Broader hand-written expert mixing was negative. Hard base guardrails reduce bridge trace-core success from 72.0% to 63.0% or 60.0%. Soft base-score mixing is inert at low mixing weights and harmful at high weights. Imported-failure multi-expert routing drops bridge trace-core success to 62.0%. A larger final expert guardrail also underperforms final-base8 on the original split, fold0, and bridge subset. These results justify stopping manual expert-mix tuning and treating calibrated expert gating as future work.